

D-Fence — Scoring Specification

Version 1.0 · 2026-09-03 · status: **PROPOSAL for team review** Satisfies requirements 4.1.4 (a normalisation method per driver) and 4.1.5 (driver weights). Implemented by `src/control/normalisation/` and `config/scoring.default.json`.

Read this first. Every number in §2 and §3 is a *judgement*, argued from evidence but not measured against dengue outcomes. They live in configuration precisely so the team can change them without touching code (10.6.2). What is *not* negotiable is the shape: seven drivers (4.1.3), each normalised to [0, 1] (4.1.4), weights summing to 1.0 (4.1.6), a 0–100 score to one decimal place (4.1.7), tiers at 40.0 and 70.0 (4.1.8).

1. The evidence these choices rest on

A live pull of the NEA Dengue Clusters GeoJSON on **2026-09-03** (dataset `d_dbfabf16158d1b0e1c420627c0819168`, 25 KB, 12 active clusters):

Cluster (truncated)	CASE_SIZE	Habitat types listed
Countryside Rd / Florissa Pk / Lentor Ave	258	28
Jln Kayu / Jln Tari Dulang	61	12
Luxus Hill Ave / Seletar Green Ave	57	11
Ho Ching Rd / Kang Ching Rd	48	16
Admiralty Lk (Blk 493)	5	0
Yishun Ave 9 / Yishun Cl	3	0
seven further clusters	2 each	0

Three facts from that payload drive everything below.

1. **Case size is extremely long-tailed.** One cluster holds 258 cases; eight hold two.
2. **The habitat fields are sparse.** Eight of twelve clusters list no habitat types at all, and `CONSTRUCTION_SITES` is null for ten of twelve.
3. **The feed moves slowly.** Only two distinct `FMEL_UPD_D` timestamps across all twelve features (25 Aug and 28 Aug) — revision on the order of days.

2. Normalisation method per driver (satisfies 4.1.4)

Driver	Method	Parameter	Class
Case size	Log scale against the observed maximum	—	LogScaleNormalisation
Case growth delta	Log scale against the observed maximum, negatives floored at 0	—	LogScaleNormalisation
24-hour rainfall	Capped linear	cap 50 mm	CappedLinearNormalisation
72-hour rainfall	Capped linear	cap 120 mm	CappedLinearNormalisation
Verified open report count	Capped linear	cap 5 reports	CappedLinearNormalisation
Days since last treatment	Linear rise to saturation	60 days	RecencyDecayNormalisation
Premises mix	Pass-through of the 1.1.15 value	—	PremisesMixNormalisation

2.1 Case size — log, not min-max

The skeleton bound case size to `MinMaxNormalisation`. Against the real payload that is wrong, and the numbers say so plainly:

Cases	Min-max	Log
258	1.000	1.000
61	0.230	0.743
57	0.215	0.731
48	0.180	0.700
5	0.012	0.322
2	0.000	0.198

Min-max lets **one** outlier define the scale and collapses the other eleven clusters into the bottom quarter of the driver — a 61-case cluster and a 2-case cluster become almost the same number, so the driver stops discriminating exactly where an Operations Manager needs it to. Log preserves order and keeps the separation. `MinMaxNormalisation` is retained in the Strategy family (and still tested); it is simply not the right method for this distribution.

2.2 Rainfall — capped, and 72 hours weighted above 24

Aedes aegypti needs standing water to persist for several days for eggs to reach adults, so rain that fell over three days is a better breeding proxy than rain in the last twenty-four hours. Both are capped because past saturation more rain does not mean more breeding — it means run-off, and an uncapped driver would let one storm outvote the other six drivers. The caps (50 mm / 120 mm) are set near the upper end of ordinary Singapore accumulations, not at record values.

2.3 Days since last treatment — 60-day saturation

This is the driver that makes 4.1.17 true: a verified treatment record must lower the score. It rises linearly and saturates at 60 days, beyond which "not treated recently" no longer separates clusters. Note the interaction with 4.1.16 — an untreated cluster defaults to 90 days, so it enters at the saturated value of 1.0. That is intended: never-treated is the worst case for this driver.

2.4 Premises mix — pass-through, per 4.1.21

1.1.15 already produces a value in [0, 1], and 4.1.21 forbids transforming it again. The strategy therefore only checks the contract and throws on an out-of-range input, rather than clamping — a silent clamp would hide an upstream defect behind a plausible-looking score.

3. Proposed default weights (satisfies 4.1.5)

```
{
  "CaseSize": 0.30,
  "CaseGrowthDelta": 0.20,
  "Rainfall72h": 0.12,
  "Rainfall24h": 0.08,
  "DaysSinceLastTreatment": 0.15,
  "VerifiedOpenReportCount": 0.10,
  "PremisesMix": 0.05
}
```

Sum = 1.000, as 4.1.6 requires.

Driver	Weight	Why this weight
Case size	0.30	The only driver that is a measured health outcome rather than a proxy. It is NEA's own severity signal, and a ranking that disagreed with it would be hard to defend to a resident.
Case growth delta	0.20	Trajectory, not level — a 40-case cluster growing by 10 needs attention before a stable 60-case one. This is the driver that makes the ranking <i>move</i> , which is also what a live demo needs to show.
Days since last treatment	0.15	Carries the feedback loop. Large enough that a completed job visibly drops a cluster down the table (4.1.17), small enough that recency alone cannot outrank real case load.
72-hour rainfall	0.12	The better breeding proxy of the two rainfall windows (§2.2).
24-hour rainfall	0.08	Kept separate rather than folded in, so a sudden downpour still registers before it enters the 72-hour window. Rainfall totals 0.20.
Verified open reports	0.10	The citizen signal, and the reason the public is in the loop at all. Deliberately below the official drivers: reports are self-selected and gameable, and only <i>verified</i> ones count (5.3.x).
Premises mix	0.05	Held low because the data is sparse, not because the idea is weak. Eight of twelve clusters carry no habitat text, so for two-thirds of the table this driver contributes zero; a large weight would systematically punish clusters merely for being under-documented.

3.1 What a reviewer should push back on

- **Case size at 0.30 plus growth at 0.20 makes half the score NEA's own numbers.** Defensible, but it means D-Fence's ranking will rarely disagree dramatically with reading the NEA page. The value we add sits in the other 0.50.
 - **Nothing here is fitted.** No dengue outcome data was used to derive these weights; they encode domain reasoning. Say so in the demo rather than being asked.
 - **Premises mix may deserve to be dropped** if the sparsity persists. Keeping it at 0.05 and reporting its contribution honestly (4.1.10) is the middle path; retiring it is a one-line configuration change.
-

4. Tier thresholds

High ≥ 70.0 , Medium 40.0–69.9, Low < 40.0 (4.1.8), read from configuration (4.1.9). A score exactly on a threshold takes the **higher** tier. These are the Lab 4 boundary-value cases (`tests/priority-scoring.test.ts` §2.2) and the reason `assignTier` is the designated test subject.

A caution the team should know about. With the weights above, a score of 70 requires several drivers to be high at once. Against the 2026-09-03 payload — where nine of twelve clusters sit at 2–5 cases — most clusters will land in Low, and the table may show one High and eleven Lows. If that holds after the first week of live data, the answer is to re-cut the thresholds against the observed score distribution, **not** to inflate the weights.

5. Degraded scores

A driver whose source is stale is excluded (4.1.12), the score is marked DEGRADED (4.1.13), every excluded driver is named (4.1.20) and the remaining weights are renormalised to 1.0 (4.1.19). A missing driver is never treated as zero: a rainfall feed that is down must not read as a dry cluster.

`PriorityScoringEngine.applyWeights()` implements the renormalisation by dividing by the weight actually present.

6. Change control

These values are configuration, not code (10.6.2). Change them in `config/scoring.default.json`; `ConfigSet.validate()` rejects any set that does not sum to 1.0 at startup, and the Lab 4 test W2/W3/W5 cases cover that rejection.